Interview prep: Tech Lead, Payments & Financial Infrastructure

Triply · Panel: HR and the technical dev team · Prepared 18 October 2026

How to use this. Every answer is written in your voice as talking points, not a script. Say it in your own words and add a real detail each time; specifics are what convince a panel. Orange boxes mark facts only you know (volumes, team sizes, dates). Fill them in tonight: a real number beats an impressive adjective. Never say anything you can't back up when the dev team asks "how exactly?".

1. Your positioning in one breath

60-second "Tell us about yourself":
"I'm a payments and core-banking engineer at NCBA Group. I build systems where money actually moves:

- EFT, RTGS and PesaLink processing;
- cheque clearing microservices for Uganda, Kenya, Rwanda and Tanzania;
- prepaid cards;
- an open banking API that lets external partners integrate with the bank;
- AML classification and automated reporting to the FRC under Kenya's AML and counter-terrorism-financing rules.

Before banking I was a machine learning engineer, working on speech recognition and RAG pipelines for speech analysis, so I'm comfortable on the data and risk side too.

This role covers exactly what I do, plus orchestration across providers and leading a team. To show how I'd approach it, I built *Mizani*: a payments and core-banking platform in Go. It has a double-entry ledger, real Daraja, KCB Buni and Equity Jenga integrations, idempotency at every layer, reconciliation and a tamper-evident audit trail, and it runs on five databases. I'd like to bring that to Triply and build it with a team."

Three strengths to keep coming back to

1. **I've run regulated money movement in production:** RTGS, EFT, PesaLink, cheque clearing in four countries, cards. I know cut-offs, returns, reconciliation and what happens when a rail misbehaves.
2. **I build for correctness and audit first:** ledgers that always balance, idempotency, validating callbacks with the provider, AML reporting a regulator accepts.
3. **I'm a builder who can lead:** I designed and shipped Mizani end to end, wrote its decision records (ADRs) and runbooks, and set up its CI and security scanning. That's the standard I'd set for a team.

The ad, mapped to your evidence

Responsibility in the ad	Your NCBA / prior experience	What Mizani shows
Payment orchestration, collections, payment links	Open banking API for external integrators; channel integrations	Routing with failover across M-Pesa, KCB, Jenga cards and acquirers; payment links; hosted checkout

Wallets, balances, ledger	Core-banking postings behind EFT, RTGS and PesaLink; prepaid card balances	Double-entry ledger, available vs held balances, T24-style GL, COB, trial balance and integrity checks
Payouts, transfers, FX, money movement	EFT, RTGS, PesaLink processing; cross-border clearing (UG/KE/RW/TZ)	Payouts via Daraja B2C, KCB and Jenga with leases and no resends; FX quote then execute
Cards and card processing	Prepaid card projects	Issuing, authorise, capture, reverse; Jenga hosted card checkout (no card numbers reach Mizani)
Settlement and reconciliation	Cheque clearing (presentment, returns, settlement); RTGS and EFT reconciliation	Settlement-file reconciliation, break resolution, automated settlement in COB
Bank, PSP and provider integrations	Open banking API; clearing houses and national rails	Live Daraja, KCB Buni and Equity Jenga integrations; callbacks are never trusted on their own
KYC/KYB, fraud, disputes, risk	AML classification and FRC reporting; ML background	KYC/KYB tiers and limits, rules-based fraud scoring, chargeback lifecycle
Production reliability and incidents	Production payment systems with cut-offs and SLAs	Circuit breakers, retries, outbox, metrics, six incident runbooks
Fin-ops tooling and engineering standards	Bank-grade controls: maker-checker, audit	Ops console, four-eyes approvals, 10 ADRs, 130 automated tests, 8 security scanners in CI

2. HR questions

HR Why do you want to leave NCBA?

> **Say:** "I'm not running away from anything. NCBA has given me depth across rails, cards, clearing and compliance. What I want next is ownership: setting the technical direction for a payments platform end to end and leading the people building it. A tech lead role at a company where payments is the product is that next step."

Keep it forward-looking. Never criticise your employer, and never share anything confidential about NCBA systems.

HR Why Triply?

> **Say:** "Three reasons. The scope: payments, wallets, cards, payouts, FX and risk in one role is rare. The stage: you're building the infrastructure now, so the decisions I'd help make will matter for years. And the mix of 80% technical and 20% leadership fits how I work. I want to stay hands-on while growing a team." Add one specific about their product: look at triply.co tonight: what they sell, who pays whom, which markets.

HR Tell us about a time you led without formal authority.

> **Structure (STAR):** pick a real cross-team effort. Good candidates are the four-country cheque clearing services (alignment across country teams) or the open banking API (partners, security and business all needed). **Situation**: what was at stake. **Task**: your part. **Action**: how you got agreement (a design doc, a working prototype, data). **Result**: a measurable outcome: go-live date, volume, defects.

HR Describe a mistake or failure and what you learned.

> **Say:** pick something real but recoverable, own it plainly, and spend most of the answer on what changed afterwards: a test, a check, a runbook, a review step. "Since then I…" is the important sentence. A good theme for this role: you trusted a status or callback that turned out to be wrong, and now you always confirm with the source. That's exactly the principle behind Mizani's callback validation.

HR How do you handle pressure, for example a production issue during a cut-off?

> **Say:** "Contain first, then fix. Stop the bleeding (pause the queue, fail over, stop retries that could double-pay), communicate early with a clear owner and update times, then fix the root cause in daylight. With money, the worst mistakes come from rushing a fix, so I protect correctness over speed." Back it with a real RTGS, EFT or clearing example your incident.

HR How do you handle disagreement with a manager or product owner?

> **Say:** "I separate the goal from the solution. Usually we agree on the goal and disagree on risk or timing, so I make the trade-off concrete: what we'd ship, what could go wrong, what it costs to fix later. Then I offer an option, such as a smaller scope with the safety controls intact. If it's decided against me, I commit fully and record the risk."

HR **Where do you see yourself in 3–5 years?**

> **Say:** "Leading payments engineering for a growing platform, with a strong team I've helped hire and develop, and systems that scale without heroics. Whether the title is head of engineering or principal engineer matters less to me than owning outcomes."

HR **Salary expectations, notice period, availability.**

> **Say:** give a researched range, not a single number: your range, KES gross per month. "I'm flexible on structure; I care about the total package and the scope." Know your NCBA notice period exactly notice period.

If asked first, it's fine to say you'd like to understand the full scope and package, then give the range.

HR **Any questions for us?** **(See section 8; always ask two or three.)**

3. Leadership questions (the 20%)

LEAD **How would you lead a small engineering team?**

> **Say:**
> - **Clarity:** a short written plan and clear owners, with ADRs for decisions that matter, so we don't re-argue them.
> - **Standards:** code review on everything that touches money, tests that prove the money rules, CI that blocks on failures, runbooks for every incident type.
> - **Growth:** regular 1:1s, pairing on hard problems, and giving people ownership of whole features, not tickets.
> - **Hands-on:** I still write code, usually the risky core pieces or the hardest integration, but I don't become the bottleneck on reviews.

LEAD **What would you do in your first 90 days?**

> **Say:** "First 30: listen and learn. Every money flow end to end, the providers, the last incidents, where reconciliation hurts, and one-to-ones with everyone. Days 30 to 60: fix the riskiest gap I find. In payments that's usually idempotency on provider calls, reconciliation or monitoring of money flows. Days 60 to 90: agree a 6-month technical roadmap with product, and put in place the team habits that keep it healthy."

LEAD **How do you handle an underperforming engineer?**

> **Say:** "Early, privately and specifically. First I find out why: unclear expectations, skills, or something personal. Then I agree concrete expectations and a short timeline, and pair with them or give them a mentor. I document it. Most cases improve with clarity and support; if not, I involve HR fairly and on time. It's unfair to the rest of the team to let it drift."

LEAD **How do you balance speed of delivery with quality?**

> **Say:** "I split the code by risk. Anything that moves money or touches the ledger gets full rigour: tests, review, idempotency, an audit trail. UI, internal tools and reports can move fast and iterate. That keeps delivery quick without gambling with customer money."

LEAD **How do you run code reviews and engineering standards?**

> **Say:** "Reviews check correctness first: concurrency, failure paths, what happens on retry. Then readability. Style is automated with formatters and linters so humans don't argue about it. In Mizani the CI runs tests on every supported database, plus SAST, secret scanning, dependency checks, container scans and DAST, all with free tools. I'd bring the same discipline at a sensible weight for the team's size."

LEAD **How do you hire?**

> **Say:** "A practical exercise close to the real work, like 'design a payout with retries', rather than puzzles. I look for people who think about failure cases unprompted, write clearly and are honest about what they don't know. Every candidate gets a consistent loop and a written debrief."

4. Technical questions: payments and core banking

TECH **How do you design a wallet ledger? Why double-entry?**

> **Say:**
> - Every movement is a balanced journal entry: debits equal credits, always. Money is never created or lost, and the books are provable.
> - Amounts are integers in minor units (cents), never floats.
> - Entries are immutable. Mistakes are corrected with a compensating entry, never an edit.
> - Each wallet has an *available* and a *held* balance (authorisations, disputes, payouts in flight).
> - Every entry has a unique reference, so a replayed posting is rejected by the database.
> - At day end a trial balance per currency and a check that stored balances equal the sum of journal lines. In Mizani COB halts if either fails.

TECH **Two debits hit the same wallet at the same time. How do you stop an overdraft?**

> **Say:** "The balance check and the posting happen in the same database transaction, with either a row lock (`SELECT … FOR UPDATE`) or optimistic concurrency (`UPDATE … WHERE version = ?`, retry on conflict). In Mizani, writes go through one serialised writer per instance, plus version checks as defence in depth. That removed the retry storms on hot accounts such as provider floats. A test fires concurrent transfers and proves the account can never go negative."

TECH **What is idempotency and how do you implement it end to end?**

> **Say:** "Doing the same request twice must have the same effect as doing it once. In practice:
> - **At the API:** clients send an `Idempotency-Key`. We store it with a fingerprint of the request and replay the stored response. The same key with a different body is an error.
> - **To providers:** a stable key per operation, such as `payout:{id}`, so a retry can't create a second transfer.
> - **In the database:** unique constraints on references and on provider transaction ids.
> - **In workers:** leases, so the API and a background retry can't both send the same payout."

TECH **You send a B2C or RTGS payment and the call times out. What do you do?**

> **Say:** "This is the most important question in payments. A timeout means **unknown**, not failed. I never retry and never fail over to another rail, because that's how you pay twice. I mark it pending, then confirm with the provider's status API (Daraja Transaction Status, the bank's enquiry) or the next statement. Only a definitive answer moves the money: paid, or reversed back to the customer. Mizani treats this 'ambiguous' outcome as its own error type that the retry layer refuses to retry, and there's a test proving a payout in flight at the deadline is never resent." Connect it to your RTGS, EFT and PesaLink experience a real example.

TECH **How do you handle provider callbacks and webhooks safely?**

> **Say:** "A callback is a trigger, not the truth. Authenticate it (signature, secret path token, mTLS or Basic, depending on the provider). Store it raw for evidence. De-duplicate by provider transaction id. Then confirm with the provider's status API before moving money. Daraja callbacks aren't signed, so in Mizani an STK callback only triggers an STK Push Query, and the query's answer credits the customer. Handle out-of-order and duplicate delivery, and run a sweeper for callbacks that never arrive."

TECH **Walk us through an M-Pesa STK Push collection.**

> **Say:** "We call the STK Push API with the amount, phone, account reference and callback URL, using a password built from the shortcode, passkey and timestamp. Safaricom returns a CheckoutRequestID; the customer gets a PIN prompt. The result arrives by callback. ResultCode 0 is success; 1032 is cancelled by the user; 1037 means the customer couldn't be reached. Because callbacks get lost or forged, I confirm with STK Push Query, and a sweeper queries anything with no callback after a couple of minutes. For paybill (C2B), use the validation URL to reject unknown account numbers before money moves, and park payments with an unknown reference in suspense rather than guessing."

TECH **EFT vs RTGS vs PesaLink: when do you use each?**

> **Say:** "RTGS (KEPSS in Kenya) settles high-value payments one by one, in real time, in central bank money: final and irrevocable, but within operating hours and with higher fees. EFT is batch-cleared through the clearing house, cheaper, with next-cycle settlement and cut-offs. PesaLink is IPSL's instant interbank rail, real time around the clock for retail-sized amounts. In an orchestrator you route on amount, urgency, cost, cut-off time and beneficiary bank, and you never fail over an ambiguous attempt." Add what you built on each rail your NCBA specifics.

TECH **Tell us about the cheque clearing work across four countries.**

> **Say:** structure it as: each country's clearing house rules and cut-offs differ, so what was shared versus configured per country? Cover the presentment, clearing and return cycle, how returns and unpaid items were handled, and how reconciliation and settlement were confirmed. Talk about the microservice boundaries you chose and why. volumes, countries' differences, your role. Lessons for Triply: country-specific rules belong in configuration and adapters, not forks of the core.

TECH **How does reconciliation work, and how do you handle breaks?**

> **Say:** "Match the provider's settlement file line by line against our succeeded charges, refunds and payouts, by provider reference, amount, currency and type. Breaks come in three kinds:
>
> - amount mismatch;
> - missing in our ledger (a lost callback or a crash);
> - missing in their report (usually a timing cut-off; it appears tomorrow).
>
> Breaks are resolved with an evidenced note, and settlement posts only the matched totals: bank, provider fees and the provider float. Differences go to suspense, which must clear. In Mizani, files dropped in an inbox are reconciled automatically in COB, and clean runs settle without anyone acting."

TECH **How do you fail over between providers without double-paying?**

> **Say:** "Fail over only on a *definite* failure, for example the provider refused, the circuit is open or it's down for maintenance. Never fail over on a timeout. A per-provider circuit breaker stops hammering a sick provider. Retries use exponential backoff with jitter and the same idempotency key. Funds stay reserved in an in-transit account while a payout is queued, so the customer can't spend them twice."

TECH **Cards: authorisation, capture, chargebacks. And how do you reduce PCI scope?**

> **Say:** "Authorisation places a hold on available funds. Capture settles the final amount, possibly less, releasing the rest. Reversal or expiry releases the hold. A chargeback moves the disputed amount to held immediately; the merchant submits evidence before a deadline; winning releases it and losing sends it back through the scheme. If there's no evidence by the deadline it's a loss, which Mizani now books automatically. For PCI scope, never touch card numbers: use a hosted checkout or tokenisation from the PSP, so you're in the smallest scope (SAQ A). 3-D Secure 2 shifts fraud liability for authenticated transactions." Tie in your prepaid card projects what you built: load, spend, limits.

TECH **How did you secure the open banking API for external users?**

> **Say:** structure it as:
>
> - authentication (OAuth2 client credentials, or mTLS for partners);
> - customer consent and scopes for account data;
> - an API gateway with rate limits and quotas;
> - idempotency on payment initiation;
> - signed webhooks to partners;
> - a sandbox with test data, versioning, onboarding docs;
> - full audit logging of who accessed what.
>
> your actual stack and what you'd improve. Link it: "Triply will integrate with providers *and* may expose APIs to partners. I've built both sides."

TECH **Tell us about the AML classification and FRC reporting.**

> **Say:**
>
> - **The obligations:** under the Proceeds of Crime and Anti-Money Laundering Act and the counter-terrorism-financing rules, reporting institutions must file suspicious transaction reports and large cash transaction reports to the Financial Reporting Centre, in the FRC's goAML XML format.
> - **What you built:** the classification logic that flags transactions, the pipeline that builds valid goAML XML from core-banking data, and automated submission with tracking.
> - **Lessons:** data quality is the hard part (customer identifiers, counterparties); reports must be reproducible and auditable; regulators reject malformed files, so validate against the schema before submitting.
>
> volumes, timelines, what you automated that was manual. Link it: "At Triply, KYC/KYB tiers, transaction monitoring and an audit trail regulators can trust are part of the same system."

Check the exact reporting thresholds and deadlines before quoting numbers; give them only if you're certain.

TECH **How would you approach fraud and risk? Where does your ML background fit?**

> **Say:** "Start with explainable rules: velocity, amount, new customer, new beneficiary, blocklists. Every decision is stored with its reasons, so ops can explain it and we gather labelled data. Mizani does this, and now re-scores held payouts automatically when signals lapse. Once there's enough labelled data, add an ML score as one more signal, not a black box, and monitor drift and false positives. From my ML work I know the hard parts are labels, class imbalance and serving latency, not the model."

TECH **KYC vs KYB, and how do tiers work?**

> **Say:** "KYC verifies individuals (ID, date of birth, sanctions and PEP screening). KYB verifies businesses: registration, directors and ultimate beneficial owners. Tiers tie verification level to limits on single transaction size and daily volume, so risk grows with assurance. Limits are enforced in the same transaction as the money movement, not in the UI."

TECH **FX: how do you quote and execute safely?**

> **Say:** "Give a firm quote with the rate, spread or fee and a short expiry (a minute), executable once. Execution posts a balanced multi-currency entry through FX position accounts, so each currency's books balance. Round explicitly in minor units and record the rate used. Reject expired or reused quotes."

TECH **Which regulations matter for a Kenyan payments platform?**

> **Say:** "The National Payment System Act and its regulations (CBK licensing and oversight of payment service providers), the AML and counter-terrorism-financing laws with FRC reporting, the Data Protection Act 2019 (registration with the ODPC, lawful processing, data minimisation), PCI DSS for cards, and the scheme rules of the providers we use. Expanding regionally adds each country's central bank and FIU rules. My four-country clearing work is directly relevant here."

5. System design (expect one whiteboard prompt)

DESIGN **"Design a payments and wallet platform: collections via M-Pesa and cards, payouts to banks and M-Pesa."**

Answer in this order (say the order out loud first):

1. **Clarify:** volumes (e.g. 10k/day now, 1M later), markets, currencies, latency, who the users are, regulatory scope.
2. **Core:** a double-entry ledger as the single source of truth; wallets with available and held balances; immutable entries.
3. **API:** idempotency keys on every money-moving call; request validation; auth per client.
4. **Orchestration:** provider adapters behind one interface; routing rules (cost, success rate, amount, rail hours); retries with backoff and a circuit breaker per provider; never fail over on ambiguity.
5. **Async truth:** callbacks authenticated, stored, de-duplicated, then confirmed by status query; a sweeper for missing callbacks.
6. **Events:** a transactional outbox, so a customer webhook is sent if and only if the posting committed; signed webhooks, at-least-once delivery, consumers de-duplicate.
7. **Risk:** limits by KYC tier and a rules engine, evaluated in the same transaction.
8. **Back office:** reconciliation and settlement, maker-checker for manual changes, an audit trail, ops console, end-of-day jobs.
9. **Operations:** metrics on money flows (success rate per provider, stuck items, suspense balance), alerts, runbooks.
10. **Scale:** start as a modular monolith on Postgres. Shard or partition by account when needed, and use queues for provider calls. Hot accounts such as provider floats are the real bottleneck; batch or sub-account them.

This is Mizani's architecture. Saying "I built this; here's what I'd keep and change at your scale" is powerful.

DESIGN **"How does it scale from 10k to 1M transactions a day?"**

Say: "1M a day averages about 12 per second, with peaks maybe ten times that. That's well within one good Postgres primary if the design is right. The real issues are hot accounts, provider rate limits and long-running provider calls holding resources. So: keep provider calls outside database transactions, use queues and worker pools per provider, split hot system accounts, add read replicas for reports, and partition journal tables by date. Mizani measured about 93 transactions a second on a laptop with full synchronous evidence logging, roughly 8M a day, so the ceiling is architecture and operations, not language."

DESIGN **"Design the payout service."**

Say: "Reserve the funds on creation (wallet to in-transit), score the risk and send to review if needed. Take a lease, call the rail with a stable idempotency key, and on success book in-transit to provider float. A definite failure reverses to the wallet. An ambiguous outcome is pending and settled by a status query. Retry with backoff when every rail is down, failing and reversing after N attempts. Send webhooks through the outbox, and reconcile against the provider's settlement report."

6. Reliability and incidents

TECH **Tell us about a production incident you owned.**

> **Structure: detect** (how you found out; ideally an alert before the customers did) → **assess** (impact in money and customers) → **mitigate** (stop further harm) → **recover** (fix the data through proper postings, never direct edits) → **follow up** (blameless postmortem, the test or alert you added).
> your real incident, e.g. from RTGS, EFT, PesaLink or clearing.

TECH **What do you monitor in a payments system?**

> **Say:** "Business signals, not just CPU:
> - success rate and latency per provider;
> - items stuck in pending past their SLA;
> - queue depth and retry rates;
> - reconciliation breaks;
> - suspense and unapplied balances;
> - webhook dead letters;
> - whether end-of-day close completed and the books balanced.
>
> Each alert links to a runbook. Mizani has runbooks for provider outage, stuck payouts, halted COB, reconciliation breaks, dead letters and unapplied deposits."

7. Questions about your project (Mizani)

TECH **Walk us through Mizani.**

> **Say (2 minutes):**
>
> - **What it is:** a Go modular monolith: a double-entry ledger with T24-style core banking (business dates, COB, GL, maker-checker), payments, payouts, cards, FX, disputes, risk, reconciliation.
> - **Integrations:** real Daraja, KCB Buni and Equity Jenga (including Jenga card checkout), validated with each provider's status APIs.
> - **Audit:** a hash-chained trail with the ledger sealed into it, append-only database triggers, and a trace of any transaction end to end.
> - **Correctness:** idempotency at the API, provider, database and worker layers.
> - **Deployment:** runs on SQLite, PostgreSQL, MySQL, SQL Server or Oracle by configuration, fully containerised.
> - **Automation:** standing orders, payout re-scoring, settlement-file reconciliation and dispute expiry.
> - **Proof:** 130 automated tests, CI across all databases, and 8 free security scanners.

TECH **Why Go? Why SQLite by default?**

> **Say:** "Go gives a single static binary, a small distroless image, fast startup, simple concurrency and explicit code that reads like the money rules. It started in .NET; under concurrent load, optimistic writers on hot accounts kept conflicting, which pushed me to a serialised-writer design. SQLite keeps the demo one container. Production is a config change to Postgres or another server database, and CI tests every engine."

TECH **What's real and what's simulated?**

> **Say, honestly:** "The Daraja, KCB Buni and Jenga integrations are real code against their documented APIs; they run live when credentials are set and simulated otherwise. The card acquirers other than Jenga, and KYC identity checks, are simulated. There are no public APIs for them, so I'd integrate the real PSP or identity provider (for example Smile ID) at Triply."

Saying this before they find it builds trust with a technical panel.

TECH **What would you change before production?**

> **Say:** "SSO with per-operator permissions for the console instead of demo keys; secrets from a vault; a real KYC provider; load and chaos testing against provider sandboxes; partitioning journal tables; and an on-call rota with alerting wired to the runbooks."

TECH **What was the hardest bug?**

> **Say:** pick one you can explain deeply:
>
> - **Reference collisions:** random transaction-reference suffixes collided under load (birthday problem: about 25k entries a day against 32^5 combinations). The fix was a per-day counter scrambled by a bijection.
> - **Timestamp precision:** audit hashes broke on databases that store only microseconds, so the clock is truncated to microseconds everywhere.
> - **Oracle empty strings:** Oracle treats an empty string as NULL, which silently changes comparisons.

8. Questions to ask them

To the dev team

- Which providers and rails do you run today, and which causes the most incidents?
- How is reconciliation done now, and how manual is it?
- What does the ledger look like today: double-entry, or balances on rows?
- What's the stack and how do you deploy? What does on-call look like?
- What was your last serious incident, and what changed after it?

To HR / the hiring manager

- How big is the team I'd lead, and what's the hiring plan?
- What would make this hire a success after 6 and 12 months?
- How are technical decisions made between engineering and product?
- What are the next steps and timeline?

9. Day-of checklist

- Have Mizani running locally (`docker compose up` or `go run ./cmd/mizani`), with the ops console and `/docs` open in tabs, ready to share your screen.
- Keep this sheet and the job ad open. Have two STAR stories ready: one incident, one leadership.
- Fill in every orange box with real numbers tonight.
- Say nothing confidential about NCBA systems, customers or internal figures. Speak in patterns and your own role.
- For any design question: clarify first, state your structure, then go deep on failure cases. That's where payments engineers are judged.
- If you don't know something, say so and reason it through aloud. That scores better than bluffing with a technical panel.